APAI2026 - LAB04

PULP_NN

Authors: Călin Diaconu, Davide Nadalini, Alberto Dequino, Francesco Conti

Contacts: calin.diaconu2@unibo.it, seyedahmad.mirsalar2@unibo.it, ihor.sachok@unibo.it

Links: GitHub Link (code)

Summary

1. Subject(s):
    - Parallelization on the PULP architecture
    - Matrix multiplication
    - Fully connected layer
    - 2D convolution
    - Profiling code execution
2. Programming Language: C
3. Lab duration: 3h
4. Assignment:
    - Time for delivery: 1 week
    - **Submission deadline: Sunday November 8th 2026 (end of day, 23:59)**

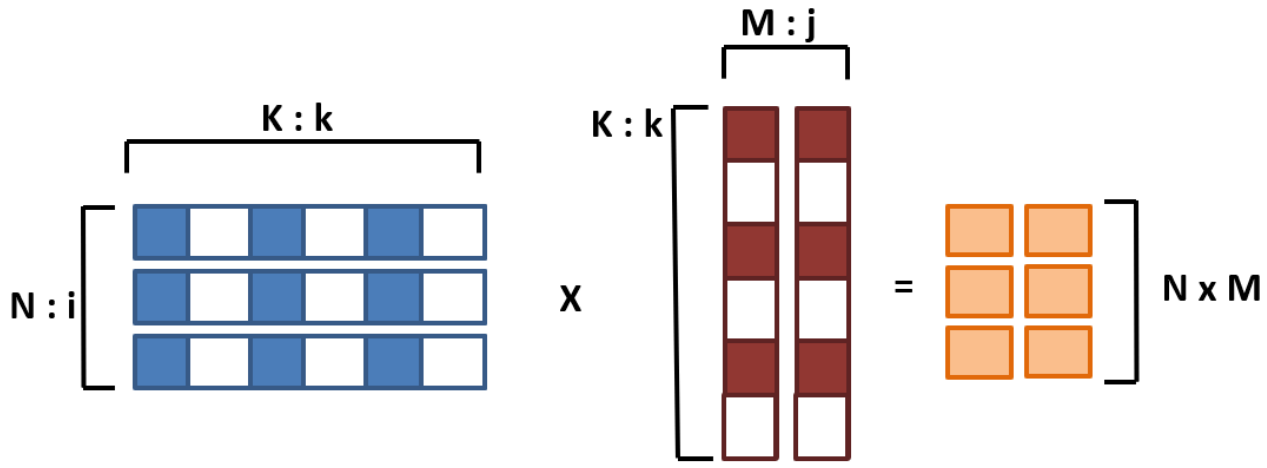
How to deliver the assignment

You will deliver ONLY THIS TEXT FILE, no code

- Download this file.
- Fill in the required results.
- Export to PDF format.
- Rename the file to: LAB<number_of_the_lesson>_APAI_<your_name>.pdf
- Use the Virtuale platform to upload ONLY your .pdf file

LAB STARTS HERE

Task 1: Matrix Multiplication



Matrix sizes: N x K * K x M = N x M
Initial sizes: N=32, M=16, K=16

1.0. Setup

- If working in the **virtual machine**, don't forget to source the SDK in new terminals: `source setup-pulp-sdk.sh`
- Go to `01_matmul_parallelization/` folder
- Every time you want to run the code, **SAVE your file** and write in the terminal: `make clean all run`

1.1. Speedup and Amdahl's law

Task Location: `01_matmul_parallelization/cluster.c`

Setup:

- CORES=<varying>
- N=32, M=16, K=16

Sub-tasks:

- Enable performance counters. You will find them in the code → uncomment them
- Fill in the table below: *Measure the execution cycles, and calculate the speedup w.r.t. a single core.*

***Tip**: for executing with different numbers of cores, use the "`CORES`" flag.*
Example: `make clean all run CORES=8`

Cores (#N)	Cycles	Speedup (w.r.t. CORES=1)
1	39748	1
2	20073	1.980
4	10466	3.797
8	6587	6.034

1.2. Explore different input sizes

Task Location: `01_matmul_parallelization/cluster.c`

Setup:

- CORES=8
- N=<varying> M=16 K=16

Sub-tasks:

- Activate the necessary counters in the code
- Measure performance of each individual core *(Instructions executed, and IPC)*

Matrix size: N	Instructions executed (each individual core)							
	0	1	2	3	4	5	6	7
4	1019	1019	1019	1019	36	36	36	36
8	1019	1019	1019	1019	1019	1019	1019	1019
80	9768	9768	9768	9768	9768	9768	9768	9768
81	10740	10740	10740	10740	10740	10740	10740	3936

Matrix size: N	IPC (each individual core)							
	0	1	2	3	4	5	6	7
4	0.61	0.61	0.61	0.61	0.02	0.02	0.02	0.02
8	0.43	0.43	0.43	0.43	0.43	0.43	0.43	0.43
80	0.72	0.72	0.72	0.72	0.72	0.72	0.72	0.72
81	0.72	0.72	0.72	0.72	0.72	0.72	0.72	0.26

Answer the following questions:

Is the workload equally balanced with N=4? Why?

No. There are 8 cores but only 4 rows, so cores 0–3 each process one row and cores 4–7 get no rows.

Is the workload equally balanced with N=8? Why?

Yes. There are 8 rows and 8 cores, so each core processes one row.

Is the IPC higher for N=80 or N=81? Why?

N=80 is better balanced, each core processes 10 rows. With N=81, cores 0–6 process 11 rows each, while core 7 processes only 4. Since the cores synchronize at the end, core 7 spends more time waiting, lowering its IPC. Both cases round to 0.72 for most cores, but core 7 is 0.26 for N=81.

Measure the overall performance of the GEMM: Cycles, MACs, MAC/cycle

Tip: *Calculate the MACs by hand*

N	MACs	Cycles (maximum value from all cores)	MAC/cycle (maximum value from all cores)
4	1024	1663	0.62
8	2048	2393	0.86
80	20480	13635	1.50
81	20736	14891	1.39

Answer the following question:

Why is the MAC/cycle lower when N=81 than when N=80?

With 8 cores and N=80, each core gets 10 rows. With N=81, the chunk size rounds up to 11 cores 0–6 get 11 rows each, while core 7 gets 4. The total cycle count is set by the slowest cores, so that uneven split makes the extra MACs take proportionally longer.

1.3. Load stalls & unrolling the MatMul

Task Location:

- `01_matmul_parallelization/cluster.c`
- `01_matmul_parallelization/matmuls.c`

Setup:

- 8 CORES
- N=32 M=16 K=16

Sub-tasks:

- Enable the performance counters of interest. We want to profile:
    - Execution cycles (total)
    - Number of instructions executed
    - Load stalls (not in the code yet: you must add this counter)

Complete the code where you find `/* YOUR CODE HERE */` with the right performance counters

```
pi_perf_conf(
    1 << PI_PERF_CYCLES
    | 1 << PI_PERF_INSTR
    // /*YOUR CODE HERE*/ //(add load stall)
);
```

Put your solution below (code)

coreID [1]: Instructions: 3936 | Cycles: 6083 | Load stalls: 1024 | IPC: 0.65 | N=32

coreID [6]: Instructions: 3936 | Cycles: 6083 | Load stalls: 1024 | IPC: 0.65 | N=32

coreID [0]: Instructions: 3936 | Cycles: 6083 | Load stalls: 1024 | IPC: 0.65 | N=32

coreID [5]: Instructions: 3936 | Cycles: 6083 | Load stalls: 1024 | IPC: 0.65 | N=32

coreID [4]: Instructions: 3936 | Cycles: 6083 | Load stalls: 1024 | IPC: 0.65 | N=32

coreID [3]: Instructions: 3936 | Cycles: 6083 | Load stalls: 1024 | IPC: 0.65 | N=32

coreID [2]: Instructions: 3936 | Cycles: 6083 | Load stalls: 1024 | IPC: 0.65 | N=32

coreID [7]: Instructions: 3936 | Cycles: 6083 | Load stalls: 1024 | IPC: 0.65 | N=32

***Tip**: Here's the full list of the performance counters*

```
typedef enum {
  PI_PERF_CYCLES        = 17, /*!< Total number of cycles (also includes the
    cycles where the core is sleeping). Be careful that this event is using a
    timer shared within the cluster, so resetting, starting or stopping it on
    one core will impact other cores of the same cluster. */
  PI_PERF_ACTIVE_CYCLES = 0,  /*!< Counts the number of cycles the core was
    active (not sleeping). */
  PI_PERF_INSTR         = 1,  /*!< Counts the number of instructions executed.
  */
  PI_PERF_LD_STALL      = 2,  /*!< Number of load data hazards. */
  PI_PERF_JR_STALL      = 3,  /*!< Number of jump register data hazards. */
  PI_PERF_IMISS         = 4,  /*!< Cycles waiting for instruction fetches, i.e.
    number of instructions wasted due to non-ideal caching. */
  PI_PERF_LD            = 5,  /*!< Number of data memory loads executed.
    Misaligned accesses are counted twice. */
  PI_PERF_ST            = 6,  /*!< Number of data memory stores executed.
    Misaligned accesses are counted twice. */
  PI_PERF_JUMP          = 7,  /*!< Number of unconditional jumps (j, jal, jr,
    jalr). */
  PI_PERF_BRANCH        = 8,  /*!< Number of branches. Counts both taken and
    not taken branches. */
  PI_PERF_BTAKEN        = 9, /*!< Number of taken branches. */
  PI_PERF_RVC           = 10, /*!< Number of compressed instructions
    executed. */
  PI_PERF_LD_EXT        = 12, /*!< Number of memory loads to EXT executed.
    Misaligned accesses are counted twice. Every non-TCDM access is considered
    external (cluster only). */
  PI_PERF_ST_EXT        = 13, /*!< Number of memory stores to EXT executed.
    Misaligned accesses are counted twice. Every non-TCDM access is considered
    external (cluster only). */
  PI_PERF_LD_EXT_CYC    = 14, /*!< Cycles used for memory loads to EXT.
  Every non-TCDM access is considered external (cluster only). */
  PI_PERF_ST_EXT_CYC    = 15, /*!< Cycles used for memory stores to EXT.
  Every non-TCDM access is considered external (cluster only). */
  PI_PERF_TCDM_CONT     = 16, /*!< Cycles wasted due to TCDM/log-interconnect
  contention (cluster only). */
} pi_perf_event_e;
```

Ref for the VIRTUAL MACHINE: `/rtos/pmsis/pmsis_api/include/pmsis/chips/default.h`
Ref for the DOCKER CONTAINER:
`/app/pulp-sdk/rtos/pmsis/pmsis_api/include/pmsis/chips/default.h`

- Implement the missing code in `gemm_unroll() (in matmuls.c)`. Then manually fix the code in order to unroll 2-4-8-16 operations.
- Now, fill in the table, comparing the naive `gemm()` vs. the unrolled version `gemm_unroll()`

	Stalls	Instructions	Cycles	IPC	MACs	MAC/cycle
Naive	1024	3936	6083	0.65	8192	1.35
Unrolled 2	2	4533	6169	0.73	8192	1.33
Unrolled 4	66	4902	6570	0.73	8192	1.21
Unrolled 8	66	6349	8728	0.73	8192	0.94
Unrolled 16	1,557	11435	16951	0.65	8192	0.48

Answer the following questions:

Compare the stalls of “naive” vs. unrolled (2-4-8). Which has more? Why?

The naive version has more, 1,024 stalls per core, compared with 2 for unrolled 2 and 66 for unrolled 4 and 8. Unrolling loads several values before using them, giving the processor other work to do while the loads complete.

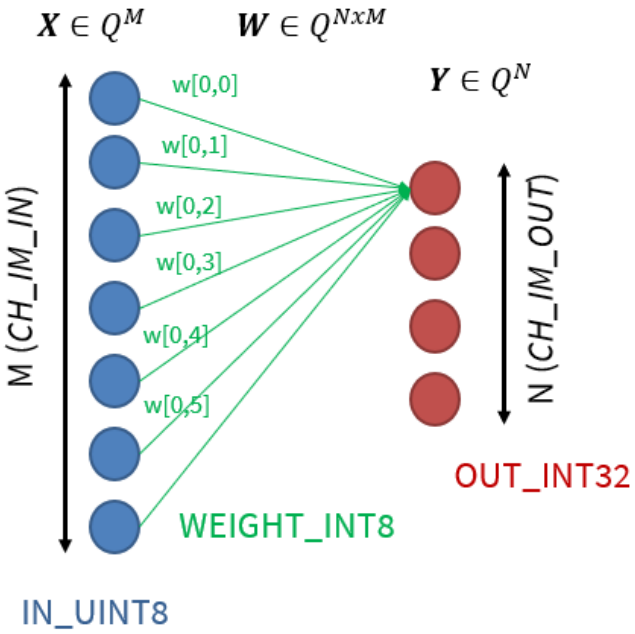
Why does Unrolled 16 have more stalls than Unrolled 8?

Unrolled 16 needs more values in registers at once. The register file is limited, so values may spill to memory, causing extra loads and stalls. The run showed about 1,548 stalls per core for unrolled 16, versus 66 for unrolled 8.

Unrolled 8 has far fewer stalls than Naive, but is it faster? Which version has the highest MAC/cycle, and why does unrolling more not keep improving it? (Hint: look at the Instructions column, not only at the stalls.)

In these runs, naive had the highest result at about 1.35 MAC/cycle. Unrolled 2 was close at 1.33, while unrolled 4, 8, and 16 measured about 1.21, 0.94 and 0.48. Unrolling reduces load stalls, but it also increases instructions and overhead. At larger factors, register spills can make the kernel slower overall.

Task 2: Fully Connected Layer



2.0. Setup

- If working in the **virtual machine**, don't forget to source the SDK in new terminals:
  `source setup-pulp-sdk.sh`
- Go to `02_fully_connected/` folder
- Every time you want to run the code, **SAVE your file** and write in the terminal:
  `make clean all run`
- Layer dimensions default to the values in *`include/data_allocation.h`* (`CH_IM_IN`=1024, `CH_IM_OUT`=16). You can override them (and the number of cores) from the command line:
  `make clean all run CORES=N CH_IN=<x> CH_OUT=<y>`
- Inputs and weights are generated at runtime and the output is checked against a reference, so any layer size is valid. If your code is correct, the program prints:
  `FullyConnected layer executed without errors.`

2.1. MACs: FullyConnected without vs. with SIMD

Implement the SIMD dot product, then compare the number of executed instructions in the normal FullyConnected layer and the SIMD version.

1. Calculate the number of MACs by hand
2. Use the performance counters to measure the number of executed instructions with the default (non-SIMD) *`dotp_u8_i8_i32`*
3. Implement *`dotp_u8_i8_i32_simd`* in *`fully_connected.c`* (see the TASK 2.1 comment there)
4. In *`fully_connected()`*, switch the call from *`dotp_u8_i8_i32`* to *`dotp_u8_i8_i32_simd`* (look for the comment "Task 2.1")
5. Run again and check that the result is still correct (the program must print "FullyConnected layer executed without errors."), then measure the number of executed instructions again

Hints for `dotp_u8_i8_i32_simd`:

- Use the *`v4u`* (4 × `uint8`) and *`v4s`* (4 × `int8`) vector types to load 4 inputs and 4 weights at a time.
- Look at the *`SumDotp4`* macro in *`include/pulp_nn_utils.h`*: it performs 4 MACs with a single SIMD instruction.
- The vector length is not always a multiple of 4: make sure the leftover elements are also processed.

Note: the number of executed MAC operations is dictated by the size of the fully connected layer, not the way operations are implemented (non-SIMD/SIMD). The formula used to calculate the total number of MAC operations is:

$$MACs = Channels_{in} * Channels_{out}$$

You can find the FullyConnected layer dimensions in the *`include/data_allocation.h`* header file (default values; see 2.0 to override them).

TIP: 8-bit SIMD instructions perform 4 MACs in 1 cycle.

Cores (#N)	SIMD	Operations (MAC)	Instructions (#N)
1	No	16384	49356
1	Yes	16384	28878

Answer the following question:

What is the ratio of instructions before/after the new SIMD implementation? Why?

The instruction ratio is before / after => 49,356 / 28,878 = 1.71. The MAC count stays the same, but SIMD handles four 8-bit MACs with one instruction, so fewer instructions are needed.

2.2. Calculate speedup

Measure the latency of a fully connected layer and fill out the table below.
From the measured latency calculate the performance and speedup relative to the single-core latency.

Note: to calculate the performance you will have to divide the total number of MAC operations by the measured latency.

Cores (#N)	SIMD	Latency (cycles)	Performance (MAC/cycle)	Speedup w.r.t. #Cores=1 SIMD=No
1	No	66073	0.25	1.0x
1	Yes	37551	0.44	1.76x
8	No	8609	1.90	7.68x
8	Yes	4978	3.29	13.27x

2.3. Leftovers and alignment

Study what happens to correctness and performance when the number of input channels is not a multiple of 4.

Setup:

- CORES=1, SIMD on (your `dotp_u8_i8_i32_simd`)
- `CH_OUT`=16
- `CH_IN`=1024, 1020, 1023, 1027

Example:

```
make clean all run CORES=1 CH_IN=1023 CH_OUT=16
```

Fill in the table below. Calculate the MACs by hand.

CH_IN	MACs	Instructions (#N)	Cycles	Performance (MAC/cycle)
1024	16384	28878	37551	0.436
1020	16320	28766	37358	0.437
1023	16368	44274	59138	0.277
1027	16432	44446	59367	0.277

Answer the following questions:

Is the result still correct for `CH_IN`=1023 and `CH_IN`=1027? Which part of your code makes it so?

Yes. Both runs printed FullyConnected layer executed without errors. The SIMD function handles leftovers with its scalar loop. When the input and weight pointers have different alignments, it safely calculates that row with the scalar fallback instead of making an unaligned SIMD load.

Compare the MAC/cycle across the four sizes. Why is it lower when `CH_IN` is not a multiple of 4? (Hint: think about the address where each weight row starts, and about the leftover loop.)

The 1024 and 1020 cases reach about 0.436–0.437 MAC/cycle, 1023 and 1027 reach about 0.277. With a non multiple of 4 channel count, each successive weight row starts at a different 4-byte alignment. Most rows therefore use the scalar fallback even aligned rows have leftover elements handled one at a time.

How could the data layout be changed to avoid the problem?

Pad the input length and each weight row's stride to a multiple of 4, filling padding bytes with zero. Then each row starts aligned and the SIMD loop can process complete groups of four.

Task 3: Convolution Layer

3.0. Setup

- If working in the **virtual machine**, don't forget to source the SDK in new terminals: `source setup-pulp-sdk.sh`
- Go to `03_convolution/` folder
- Every time you want to run the code, **SAVE your file** and write in the terminal: `make clean all run`

3.1. Speedup over multiple cores

Measure the latency of a Convolution layer and fill out the table below. From the measured latency calculate the performance and speedup relative to the single-core latency.

Note: to calculate the performance you will have to divide the total number of MAC operations by the measured latency. The formula to calculate the total number of MAC operations is:

$$MACs = Kernel\,Height * Kernel\,Width * Channels_{in} * Height_{out} * Width_{out} * Channels_{out}$$

You can find the Convolution layer dimensions in the *include/data_allocation.h* header file.

Cores (#N)	MACs	Latency (cycles)	Performance (MAC/cycle)	Speedup w.r.t. #Cores=1
1	4,718,592	2,348,320	2.01	1.0x
2	4,718,592	1,177,735	4.01	1.99x
4	4,718,592	594,953	7.93	3.95x
8	4,718,592	304,675	15.49	7.71x

3.2. Where does the time go: im2col vs matmul

Measure how the convolution time is split between its two phases.

The convolution kernel works in two phases. First, im2col copies the 3x3xCH_IN input patch of an output pixel into a contiguous buffer (two output pixels at a time). Then, the matmul kernel multiplies this buffer with the weights.

Setup:

- In *03_convolution/Makefile*, uncomment the following line (remove the leading #):
  `#APP_CFLAGS += -DPROFILE_SPLIT`
- Each core now prints its im2col cycles and its matmul cycles.
- Run with CORES=1 and CORES=8. Report the values of core 0, together with the total latency of the layer.

Fill in the table below. im2col share (%) = im2col cycles / total latency × 100.

Cores (#N)	im2col cycles (core 0)	matmul cycles (core 0)	Total latency (cycles)	im2col share (%)
1	104,700	2,248,715	2,361,240	4.43%
8	16,759	288,168	307,034	5.46%

Answer the following questions:

What fraction of the time is spent in im2col? Is the im2col parallelized?

Core 0's im2col share was 4.43% with 1 core and 5.46% with 8 cores. Yes, im2col is parallelized: each core processes part of the output pixels and builds its own im2col data.

Using Amdahl's law: if you made the matmul 2x faster, what would the overall speedup be?

Using the 1-core measurement, matmul takes about 95.2% of total latency. Amdahl's law gives an overall speedup of about 1.91× the remaining im2col work and overhead prevent the whole layer from becoming 2× faster.

Why do the im2col and matmul cycles not add up to the total latency?

The phase counters only include time recorded inside those phases total latency also includes setup and synchronization overhead. With multiple cores, the phase counts are for core 0, while total latency is the wall-clock time for the whole parallel workload.

3.3. Non-power-of-2 number of cores

Measure how the convolution scales when the number of cores is not a power of 2.

Setup:

- CORES = 3, 5, 6, 7 (reuse your results from 3.1 for CORES = 1, 4 and 8)
- Measure the latency with `PROFILE_SPLIT` commented out, as in 3.1.
- Then enable `PROFILE_SPLIT` to count how many cores do work: idle cores report 0 cycles.

Example:

```
make clean all run CORES=6
```

Cores (#N)	Latency (cycles)	Speedup w.r.t. #Cores=1	Cores doing work (#N)
3	1,177,295	1.99×	2
4	594,953	3.95×	4

5	594,731	3.95×	4
6	595,002	3.95×	4
7	595,003	3.95×	4
8	304,675	7.71×	8

Answer the following questions:

Why is the latency with 6 cores almost the same as with 4?

The kernel uses log2(6), which returns 2. It then calculates a chunk of 16 >> 2 = 4 output rows per core. Only cores 0–3 get rows, cores 4 and 5 are idle. That is why 6 cores perform similarly to 4.

Find the reason in *pulp_nn_conv_u8_u8_i8.c* (look at how `extra_chunk`, `Log2Core` and `chunk` are computed; note that with 6 cores the code takes the else branch of the if: what does log2(6) return?). How would you change the work split so that all cores get work?

Partition the 16 output rows using division and remainder instead of a power o -two chunk. With 6 cores, four cores could get 3 rows and two cores could get 2. This gives every core work.

Would your change also make 5 cores faster than 4? Why? (Hint: how many output rows are there to split?)

Not necessarily. There are only 16 output rows: with 4 cores, each gets 4, with 5 one core still needs 4 rows while the other four get 3. The longest per-core workload remains 4 rows, so latency may be similar and extra scheduling overhead could offset any gain.